E-Commerce Shopping Cart System Using Doubly Linked List

Dynamic Memory Management, Bidirectional Pointer Manipulation, and Robust Input Validation in C++

SUBMITTED BY	COURSE / SUBJECT	SOURCE FILE	ACADEMIC SESSION
Vaishali Rathod	Data Structures (C++)	Doubly_Linked_List.cpp	2026 – 2027

01 TOPIC

Project Title: Design and Implementation of a Dynamic E-Commerce Shopping Cart Using Doubly Linked List in C++.

This project demonstrates the practical implementation of a dynamic linear data structure—a **Doubly Linked List**—to manage shopping-cart operations in a console-based e-commerce environment. Each node stores a product and two links: `prev` to the previous node and `next` to the next node. The implementation uses Object-Oriented Programming (OOP), dynamic allocation, controlled ownership, and input validation.

02 PROBLEM STATEMENT

Customers may add, view, update, and remove products while shopping. The number of products is not known in advance, so fixed-size arrays impose capacity limits or waste memory. Removing an array element may also require shifting later elements. A dynamic linked structure avoids fixed capacity and supports node-level insertion and deletion.

The system therefore needs to allocate and release product nodes as required, maintain unique product IDs, validate price and quantity, recover from malformed console input, keep both forward and backward links consistent, and release all allocated memory when the cart object is destroyed.

03 OBJECTIVES

- **Dynamic node allocation:** Store product ID, name, price, quantity, `prev`, and `next` in each node.
- **CRUD cart operations:** Add items, find products, update quantity, remove products, and display itemized totals.
- **Bidirectional traversal:** Support movement toward the next and previous nodes, including reverse cart display.
- **Link integrity:** Update both neighboring links during insertion and deletion.
- **Memory safety:** Delete all nodes in the destructor and disable unsafe shallow copying.
- **Input validation:** Reject duplicate IDs, invalid IDs, negative prices, non-positive quantities, and non-numeric input.

Class / Method Overview

Class / Method	Signature / Members	Responsibility	Complexity
Node	id, name, price, qty, prev, next	Stores product details and two links.	O(1) space
findItem()	Node* findItem(int id) const	Searches for an ID from the head.	O(n) time
addItem()	void addItem(id, name, price, qty)	Validates and appends a new node at tail.	O(n) time
updateQuantity()	void updateQuantity(id, qty)	Finds a product and changes its quantity.	O(n) time
removeItem()	void removeItem(int id)	Unlinks a matching node using both links.	O(n) search
displayCart()	void displayCart() const	Traverses forward and calculates total cost.	O(n) time
displayReverse()	void displayReverse() const	Traverses backward from tail to head.	O(n) time

04 STEP-BY-STEP ALGORITHMS

Algorithm 1: Add Item to Cart (addItem)

1. Receive product ID, name, price, and quantity.
2. Search for the ID; if it already exists, show an error and stop.
3. Validate that ID is positive, name is not empty, price is non-negative, and quantity is positive.
4. Create a new node with `prev = tail` and `next = nullptr`.
5. If the list is empty, set both `head` and `tail` to the new node. Otherwise set `tail->next` to the new node and update `tail`.
6. Confirm that the item was added.

Algorithm 2: Remove Item from Cart (removeItem)

1. If the list is empty, report that the cart is empty.
2. Find the node whose ID matches the target.
3. If no matching node exists, report “Product not found.”
4. If the node has a previous node, set `prev->next = next`; otherwise move `head` to the next node.
5. If the node has a next node, set `next->prev = prev`; otherwise move `tail` to the previous node.
6. Delete the node and confirm removal.

Algorithm 3: Update Quantity (updateQuantity)

1. Read the target product ID and new quantity.
2. If quantity is not greater than zero, display an error and stop.
3. Search for the product ID.
4. If found, update its quantity; otherwise report that the product is not in the cart.

Algorithm 4: Display Cart and Total (displayCart)

1. If `head == nullptr`, display “Cart is empty.”
2. Start at `head` and set total to zero.
3. For each node, display product ID, name, price, and quantity; add `price × quantity` to the total; move to `next`.
4. Display the final total with two decimal places.

Algorithm 5: Display Cart in Reverse (displayReverse)

1. If `tail == nullptr`, display “Cart is empty.”
2. Start at `tail`.
3. Display each product and move to `prev` until the pointer becomes null.

05 SYSTEM ARCHITECTURE & POINTER FLOW

Menu flow: START → Show menu → Read and validate choice → Execute Add / Update / Remove / Display / Reverse Display → Return to menu → Exit when choice 6 is selected.

Doubly linked node layout:

head				**tail**
↓				↓
nullptr ← [101 | Laptop]	■	[102 | Mouse]	■	[103 | Keyboard] → nullptr

Pointer invariants: For every node, `node->next->prev == node` whenever a next node exists, and `node->prev->next == node` whenever a previous node exists. The head's previous pointer and tail's next pointer remain null.

06 SOURCE CODE IMPLEMENTATION (PART 1: DATA STRUCTURES)

The implementation below uses a Node class with both `prev` and `next` pointers. ShoppingCart maintains head and tail pointers so the list can be traversed in either direction and appended at the tail in O(1) time after validation.

```
#include <iostream>
#include <string>
#include <limits>
#include <iomanip>
using namespace std;

class Node {
public:
    int productId;
    string productName;
    double price;
    int quantity;
    Node* prev;
    Node* next;

    Node(int id, const string& name, double p, int q)
        : productId(id), productName(name), price(p), quantity(q),
          prev(nullptr), next(nullptr) {}
};

class ShoppingCart {
private:
    Node* head;
    Node* tail;

    Node* findItem(int id) const {
        Node* temp = head;
        while (temp != nullptr) {
            if (temp->productId == id) return temp;
            temp = temp->next;
        }
        return nullptr;
    }

public:
    ShoppingCart() : head(nullptr), tail(nullptr) {}

    ~ShoppingCart() {
        Node* current = head;
        while (current != nullptr) {
            Node* nextNode = current->next;
            delete current;
            current = nextNode;
        }
    }

    ShoppingCart(const ShoppingCart&) = delete;
    ShoppingCart& operator=(const ShoppingCart&) = delete;

    void addItem(int id, const string& name, double price, int quantity) {
        if (findItem(id) != nullptr) {
            cout << "A product with this ID already exists.\n";
            return;
        }
        if (id <= 0 || name.empty() || price < 0 || quantity <= 0) {
            cout << "Invalid product details.\n";
            return;
        }

        Node* newNode = new Node(id, name, price, quantity);
        if (head == nullptr) {
            head = tail = newNode;
        } else {
            newNode->prev = tail;
            tail->next = newNode;
            tail = newNode;
        }
        cout << "Item added to cart.\n";
    }

    void updateQuantity(int id, int newQuantity) {
        if (newQuantity <= 0) {
            cout << "Quantity must be greater than zero.\n";
            return;
        }
        Node* item = findItem(id);
        if (item != nullptr) {
            item->quantity = newQuantity;
            cout << "Quantity updated.\n";
        } else {
            cout << "Product not found in cart.\n";
        }
```

```
}

void removeItem(int id) {
    Node* current = findItem(id);
    if (current == nullptr) {
        cout << (head == nullptr ? "Cart is empty.\n" : "Product not found.\n");
        return;
    }

    if (current->prev != nullptr)
        current->prev->next = current->next;
    else
        head = current->next;

    if (current->next != nullptr)
        current->next->prev = current->prev;
    else
        tail = current->prev;

    delete current;
    cout << "Item removed.\n";
}
```

06 SOURCE CODE IMPLEMENTATION (PART 2: DISPLAY & DRIVER)

```
    void displayCart() const {
        if (head == nullptr) {
            cout << "Cart is empty.\n";
            return;
        }
        const Node* temp = head;
        double total = 0.0;
        cout << "\n--- Cart Details (Forward) ---\n" << fixed << setprecision(2);
        while (temp != nullptr) {
            cout << "ID: " << temp->productId
                 << " | Name: " << temp->productName
                 << " | Price: Rs " << temp->price
                 << " | Qty: " << temp->quantity << '\n';
            total += temp->price * temp->quantity;
            temp = temp->next;
        }
        cout << "Total Cost: Rs " << total << "\n\n";
    }

    void displayReverse() const {
        if (tail == nullptr) {
            cout << "Cart is empty.\n";
            return;
        }
        const Node* temp = tail;
        cout << "\n--- Cart Details (Reverse) ---\n";
        while (temp != nullptr) {
            cout << "ID: " << temp->productId
                 << " | Name: " << temp->productName << '\n';
            temp = temp->prev;
        }
        cout << '\n';
    }
};

int main() {
    ShoppingCart cart;
    int choice = 0;

    do {
        cout << "===== E-Commerce Shopping Cart =====\n"
             << "1. Add Item\n2. Update Quantity\n3. Remove Item\n"
             << "4. Display Cart\n5. Display Cart in Reverse\n6. Exit\n"
             << "Choice: ";

        if (!(cin >> choice)) {
            cout << "Invalid input. Enter a number from 1 to 6.\n";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            continue;
        }

        switch (choice) {
        case 1: {
            int id, qty;
            double price;
            string name;
            cout << "ID: ";
            if (!(cin >> id)) {
                cout << "Invalid ID.\n"; cin.clear();
                cin.ignore(numeric_limits<streamsize>::max(), '\n'); break;
            }
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Name: ";
            getline(cin, name);
            cout << "Price: ";
            if (!(cin >> price)) {
                cout << "Invalid price.\n"; cin.clear();
                cin.ignore(numeric_limits<streamsize>::max(), '\n'); break;
            }
            cout << "Qty: ";
            if (!(cin >> qty)) {
                cout << "Invalid quantity.\n"; cin.clear();
                cin.ignore(numeric_limits<streamsize>::max(), '\n'); break;
            }
            cart.addItem(id, name, price, qty);
            break;
        }
        case 2: {
            int id, qty;
            cout << "ID: ";
            if (!(cin >> id)) {
                cout << "Invalid ID.\n"; cin.clear();
                cin.ignore(numeric_limits<streamsize>::max(), '\n'); break;
            }
            cout << "New Qty: ";
```

```
            if (!(cin >> qty)) {
                cout << "Invalid quantity.\n"; cin.clear();
                cin.ignore(numeric_limits<streamsize>::max(), '\n'); break;
            }
            cart.updateQuantity(id, qty);
            break;
        }
        case 3: {
            int id;
            cout << "ID: ";
            if (!(cin >> id)) {
                cout << "Invalid ID.\n"; cin.clear();
                cin.ignore(numeric_limits<streamsize>::max(), '\n'); break;
            }
            cart.removeItem(id);
            break;
        }
        case 4: cart.displayCart(); break;
        case 5: cart.displayReverse(); break;
        case 6: cout << "Exiting...\n"; break;
        default: cout << "Invalid choice. Select 1 to 6.\n";
        }
        cout << '\n';
    } while (choice != 6);

    return 0;
}
```

07 SAMPLE EXECUTION & TEST CASES

Example workflow: Add Laptop (ID 101, price Rs 55000, quantity 1); add Mouse (ID 102, price Rs 750.50, quantity 2); display cart; display in reverse; update quantity for ID 102 to 3; remove ID 101; display cart again.

Test ID	Scenario	Input / Action	Expected behavior
TC-01	Multi-word name	Wireless Mechanical Keyboard	Name is captured using getline().
TC-02	Duplicate ID	Add ID 101 twice	Second insertion is rejected.
TC-03	Invalid price/quantity	Price -50 or quantity 0	Invalid details are rejected.
TC-04	Bad menu input	Choice abc	Input stream is cleared and recovered.
TC-05	Reverse traversal	Choose reverse display	Items are shown from tail to head.
TC-06	Delete middle/head/tail	Remove an existing ID	Both neighboring links and head/tail are updated.

08 CONCLUSION & FUTURE SCOPE

The E-Commerce Shopping Cart System using a Doubly Linked List demonstrates dynamic memory management and bidirectional traversal in C++. Compared with a singly linked list, each node also stores a previous pointer, making backward traversal direct and allowing deletion to reconnect adjacent nodes using the target node's two links. The implementation validates product input, prevents duplicate IDs, calculates billing totals, supports reverse display, and releases nodes through the destructor.

Algorithmic performance: Search and quantity update take O(n) time. Appending is O(n) overall here because duplicate-ID validation searches the list first, followed by O(1) tail linking. Removal is O(n) to find the item and O(1) to unlink it once found. Forward display, reverse display, and destructor cleanup are O(n). These operations use O(1) auxiliary space, excluding the list nodes.

Future enhancements: Save cart contents to a file, add product-category filters, implement price sorting, or use a hash map to accelerate ID lookup.

Compilation: g++ -std=c++17 -Wall -Wextra Doubly_Linked_List.cpp -o cart